

gitit

A natural-language interface for Git

Product abstract - August 2026

Git is foundational to modern software development, but its interface asks people to remember a large, irregular command vocabulary precisely when they are trying to focus on their work. Even experienced developers routinely pause to look up branch syntax, recall the order of rebase arguments, compare reset and restore, or reconstruct a command they used weeks ago. The cost of each interruption is small; the accumulated friction, hesitation, and risk are not.

gitit is a command-line interactive tool that turns plain-language intent into an accurate Git workflow. The name combines *git* with *get it*: users describe the outcome they want, and gitit gets them to the right command quickly, transparently, and safely.

```
$ gitit undo my last commit but keep all my changes
```

1. `git reset --soft HEAD~1`
Removes the last commit while keeping its changes staged.
2. `git reset HEAD~1`
Removes the last commit and keeps its changes unstaged.

Choose [1-2] [e] Edit [c] Copy [q] Cancel

Instead of expecting the user to translate intent into syntax, gitit accepts requests such as:

- `gitit switch to main branch`
- `gitit rebase current branch`
- `gitit pull from main remote`
- `gitit show me diff b/w branch a and main`
- `gitit create a pull request`

The first response is an executable command, followed immediately by a one-sentence explanation of what it will do. Nothing state-changing runs without confirmation. If a request maps to several plausible operations, gitit presents a short ranked list, explains the practical difference between the choices, and lets the user select, edit, copy, or cancel. Ambiguity becomes a compact decision rather than a guess.

Product experience

Speed is the defining requirement. For common operations, the interval between pressing Enter and seeing a useful suggestion should feel instantaneous. The target is sub-300 ms for locally resolved intents and a fast streamed response

for model-assisted requests. An interactive session should remain warm so that follow-up requests can reuse repository context without repeated startup or model costs.

gitit combines the user's request with read-only repository facts: current branch, remotes, working-tree state, available branches and tags, upstream configuration, and installed companion tools such as the GitHub CLI. This context allows `pull from main remote` to be interpreted against the actual repository rather than generic Git syntax. Repository inspection is separate from execution and never mutates state.

The interaction model follows four principles:

1. **Intent first.** Users state the desired outcome in their own words.
2. **Command visible.** gitit always shows the exact command before execution.
3. **Meaning explained.** Every suggestion includes a concise, outcome-oriented explanation.
4. **Risk proportional.** Read-only actions can be fast; destructive or irreversible actions receive stronger warnings and explicit confirmation.

Commands are classified by risk. Viewing history or diffs is low risk. Switching with local changes, rewriting commits, force-pushing, deleting branches, or discarding work triggers additional checks. Suggestions use structured arguments rather than shell-formatted free text, preventing accidental command chaining and unsafe interpolation. Secrets and sensitive file contents are excluded from model context by default.

Phase 1 - Useful now

The first release assumes internet access and an available hosted language model, while keeping the system deterministic wherever possible. A fast local resolver handles high-frequency intents from a curated command catalog. It recognizes common paraphrases, extracts entities such as branches and remotes, validates them against repository state, and returns a structured command candidate. The hosted model handles the long tail: uncommon phrasing, multi-step workflows, and requests that require clarification.

The model does not return an arbitrary shell string. It produces a constrained plan conforming to a schema: intent, executable, argument list, explanation, confidence, risk level, prerequisites, and optional alternatives. A policy layer validates the plan against an allowlist of supported Git and GitHub CLI operations, checks repository facts, rejects unsafe syntax, and decides what confirmation is required. Execution uses direct process invocation, not a shell.

The initial command surface should cover the workflows developers reach for most often:

- status, log, show, diff, blame, and repository inspection;
- branch creation, switching, tracking, renaming, and deletion;
- add, restore, commit, amend, stash, and unstage;
- fetch, pull, push, upstreams, and remotes;
- merge, rebase, cherry-pick, revert, and conflict guidance;
- tags and releases; and
- pull-request creation and inspection through `gh` when available.

Latency is protected with a layered pipeline: exact and semantic cache, local intent routing, parallel repository inspection, compact prompts, small response schemas, and model fallback only when necessary. The resolver records anonymous opt-in feedback such as accepted candidate, edited command, rejected suggestion, latency, and failure category. It never uploads repository contents or command history without clear consent.

Success in Phase 1 is measured by time-to-suggestion, top-one acceptance rate, safe cancellation rate, command execution success, and the share of requests resolved without a network call. A strong early target is a useful first suggestion in under one second at the 95th percentile for catalogued intents, with no unconfirmed state-changing execution.

Phase 2 - Local Git intelligence

The second phase removes the dependency on continuous internet access and general-purpose hosted models. `gitit` will adopt a compact open-weight model that can be downloaded and run locally. The model will be selected through benchmarks for intent accuracy, structured-output reliability, latency, memory footprint, license compatibility, and performance across common developer hardware.

Training data will pair diverse natural-language requests with validated, platform-aware Git plans. It will include paraphrases, misspellings, shorthand, ambiguous requests, repository-state fixtures, risk labels, explanations, clarification questions, and counterexamples that should be rejected. Synthetic generation can expand coverage, but every command family will be grounded in Git documentation and tested in disposable repositories. Opt-in, privacy-preserving usage feedback can improve real-world language coverage.

Rather than training a model from scratch, the practical path is supervised fine-tuning of an existing open-weight base, followed by preference optimization for ranking and concise explanations. Model outputs remain constrained by the same schema, validator, risk policy, and execution boundary used in Phase 1. Deterministic resolution continues to serve common commands; the local model specializes in ambiguity, paraphrase, and workflow composition.

Evaluation is behavioral, not merely linguistic. A test harness creates repositories in known states, asks thousands of intent variants, executes approved plans in sandboxes, and verifies resulting commits, branches, files, remotes, and exit

codes. Regression suites emphasize destructive edge cases and ensure that lower latency never weakens safety.

Vision

gitit is not a replacement for Git; it is a more humane way to reach Git. It preserves the power, composability, and visibility of the underlying tool while removing rote syntax recall from the critical path. Beginners gain confidence because they can see and understand each command. Experienced developers move faster because obscure workflows become available without context switching. Teams gain a consistent safety layer without hiding what actually runs.

The long-term opportunity extends beyond one-command translation. gitit can explain repository state, guide conflict resolution, compose reviewable multi-step plans, remember project conventions locally, and teach Git incidentally through repeated use. Its core promise remains simple: say what you mean, see what will happen, and stay in control.